Team Project & Task Management System

Backend code changes: exact files, additions, replacements, and verification

Prepared against the supplied Spring Boot WebFlux / R2DBC backend source. The guide covers the screenshot's 400 pagination error, the recommended task-loading endpoint, and the signup-request → admin approve/reject workflow.

Important: This is an implementation guide, not a claim that your backend has already been modified. Make a backup / Git commit first. Apply the required signup changes before testing signup UI. The task aggregation endpoint is an optional performance improvement that requires a project-service change as well.

What the screenshot proves

The task service's `Params.page(...)` rejects `size` values outside 1–100. The frontend error `/tasks/project/2 → 400: size must be between 1 and 100` is caused by requesting `size=200`. Keep the frontend page size at 100 or lower. Do not increase the backend limit unless you intentionally change its pagination policy.

Current backend facts confirmed from source: `UserRouter` has no public signup route; `POST /admin/users` is admin-only; gateway public paths are currently login/refresh/actuator; task routes include `GET /tasks/my` and `GET /tasks/project/{projectId}`; page size is capped at 100.

Implementation order

1. Fix/verify frontend task request size ≤ 100 (no backend change required).

2. Add the registration request table, entity, DTOs, repository, service methods, and routes in user-service.

3. Permit only POST /users/registration-requests through the API Gateway without JWT. Keep admin review routes protected.

4. Add a task visibility aggregation endpoint only if you want to remove the frontend's per-project task requests.

5. Build each affected service, run tests, and test with non-admin/admin tokens.

Part A — Signup request and admin approval

Expected flow: a visitor submits name, email, password, and optional phone/department; the user-service stores a PENDING request with a BCrypt hash; an admin lists pending requests and approves or rejects; approval creates an active TEAM_MEMBER account using the already-hashed password; rejection changes only the request status. Never accept a role from public signup. Never return the stored password hash in any response.

A1. Add the database table

File: `user-service/src/main/resources/schema.sql`. Append this SQL after the existing users/index definitions. The service's DatabaseConfig executes schema.sql at startup.

```
CREATE TABLE IF NOT EXISTS registration_requests (
    id BIGINT NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    password_hash VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    department VARCHAR(100),
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    requested_at DATETIME(6) NOT NULL,
    reviewed_at DATETIME(6),
    reviewed_by BIGINT,
    rejection_reason VARCHAR(500),
    PRIMARY KEY (id),
    CONSTRAINT uk_registration_requests_email UNIQUE (email)
);
CREATE INDEX idx_registration_requests_status
    ON registration_requests (status, requested_at);
```

If you want rejected users to be allowed to submit again, do not use a permanent unique constraint on email across all request history. In that case, enforce "one PENDING request per email" in service logic and use a transaction/locking strategy or a separate current-request table. The simple schema above intentionally allows one request record per email until it is reviewed; after rejection, either delete that row or adapt the constraint to your chosen history policy.

A2. Add entity

New file: `user-service/src/main/java/com/hartford/tms/user/entity/RegistrationRequest.java`

```
package com.hartford.tms.user.entity;

import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Column;
import org.springframework.data.relational.core.mapping.Table;
import java.time.LocalDateTime;

@Data @Builder @NoArgsConstructor @AllArgsConstructor
@Table("registration_requests")
public class RegistrationRequest {
    @Id private Long id;
    private String name;
    private String email;
    @Column("password_hash") private String passwordHash;
    private String phone;
    private String department;
    private String status;
    @Column("requested_at") private LocalDateTime requestedAt;
    @Column("reviewed_at") private LocalDateTime reviewedAt;
    @Column("reviewed_by") private Long reviewedBy;
    @Column("rejection_reason") private String rejectionReason;
}
```

A3. Add repository

New file: `user-service/src/main/java/com/hartford/tms/user/repo/RegistrationRequestRepo.java`

```
package com.hartford.tms.user.repo;

import com.hartford.tms.user.entity.RegistrationRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

public interface RegistrationRequestRepo
        extends ReactiveCrudRepository<RegistrationRequest, Long> {
    Mono<Boolean> existsByEmail(String email);
    Flux<RegistrationRequest> findByStatusOrderByRequestedAtAsc(
        String status, Pageable pageable);
    Mono<Boolean> existsByEmailAndStatus(String email, String status);
}
```

Part A — DTOs and service logic

A4. Public request DTO

New file: `user-service/src/main/java/com/hartford/tms/user/dto/RegistrationRequestCreate.java`. Public clients must not supply role, status, reviewer, or account-active fields.

```
package com.hartford.tms.user.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegistrationRequestCreate(
    @NotBlank @Size(max = 100) String name,
    @NotBlank @Email @Size(max = 150) String email,
    @NotBlank @Size(min = 8, max = 64) String password,
    @Size(max = 20) String phone,
    @Size(max = 100) String department
) {}
```

A5. Safe response DTO

New file: `user-service/src/main/java/com/hartford/tms/user/dto/RegistrationRequestResponse.java`. It deliberately excludes passwordHash.

```
package com.hartford.tms.user.dto;

import com.hartford.tms.user.entity.RegistrationRequest;
import java.time.LocalDateTime;

public record RegistrationRequestResponse(
    Long id, String name, String email, String phone, String department,
    String status, LocalDateTime requestedAt, LocalDateTime reviewedAt,
    Long reviewedBy, String rejectionReason
) {
    public static RegistrationRequestResponse from(RegistrationRequest r) {
        return new RegistrationRequestResponse(r.getId(), r.getName(), r.getEmail(),
            r.getPhone(), r.getDepartment(), r.getStatus(), r.getRequestedAt(),
            r.getReviewedAt(), r.getReviewedBy(), r.getRejectionReason());
    }
}
```

A6. Add methods to UserService

File: `user-service/src/main/java/com/hartford/tms/user/service/UserService.java`. Add imports for RegistrationRequest, RegistrationRequestRepo, RegistrationRequestCreate, RegistrationRequestResponse, and Flux. Add a final/Autowired RegistrationRequestRepo field consistent with the style already used in this class. Then add the methods below.

```
// Add field inside UserService:
@Autowired
private RegistrationRequestRepo registrationRequestRepo;

public Mono<RegistrationRequestResponse> requestRegistration(RegistrationRequestCreate body) {
    String email = body.email().trim().toLowerCase();
    return userRepo.existsByEmail(email).flatMap(userExists -> {
        if (userExists) return Mono.error(new ConflictException("Email already registered"));
        return registrationRequestRepo.existsByEmail(email).flatMap(requestExists -> {
            if (requestExists) return Mono.error(new ConflictException(
                "A registration request already exists for this email"));
            RegistrationRequest request = RegistrationRequest.builder()
                .name(body.name().trim()).email(email)
                .passwordHash(passwordEncoder.encode(body.password()))
                .phone(body.phone()).department(body.department())
                .status("PENDING").requestedAt(LocalDateTime.now()).build();
            return registrationRequestRepo.save(request)
                .map(RegistrationRequestResponse::from);
        });
    });
}

public Flux<RegistrationRequestResponse> pendingRegistrationRequests(int page, int size) {
    // Clamp/validate page and size in handler. Use Pageable to avoid unbounded result sets.
    return registrationRequestRepo.findByStatusOrderByRequestedAtAsc(
        "PENDING", org.springframework.data.domain.PageRequest.of(page, size))
        .map(RegistrationRequestResponse::from);
}

public Mono<RegistrationRequestResponse> reviewRegistrationRequest(
        Long requestId, boolean approve, String rejectionReason, Long adminId) {
    return registrationRequestRepo.findById(requestId)
        .switchIfEmpty(Mono.error(new ResourceNotFoundException("Registration request not found")))
        .flatMap(request -> {
```

```
            if (!"PENDING".equals(request.getStatus()))
                return Mono.error(new BadRequestException("Request has already been reviewed"));
            return userRepo.existsByEmail(request.getEmail()).flatMap(exists -> {
                if (approve && exists)
                    return Mono.error(new ConflictException("A user with this email already exists"));
                Mono<?> createAccount = Mono.empty();
                if (approve) {
                    LocalDateTime now = LocalDateTime.now();
                    User user = User.builder().name(request.getName()).email(request.getEmail())
                        .password(request.getPasswordHash()) // already BCrypt-hashed: do NOT encode again
                        .role("TEAM_MEMBER").phone(request.getPhone())
                        .department(request.getDepartment()).active(true)
                        .createdAt(now).updatedAt(now).build();
                    createAccount = userRepo.save(user);
                }
                request.setStatus(approve ? "APPROVED" : "REJECTED");
                request.setReviewedAt(LocalDateTime.now());
                request.setReviewedBy(adminId);
                request.setRejectionReason(approve ? null : rejectionReason);
                return createAccount.then(registrationRequestRepo.save(request))
                    .map(saved -> RegistrationRequestResponse.from(request));
            });
        });
}
```

Reactive caveat: this sample follows the project's Mono/Flux style, but the approval operation spans two tables. For production-grade atomicity, add a reactive transaction manager and annotate/use a reactive transaction around user creation + request status update. Also enforce email uniqueness in the database and handle a race between simultaneous submissions.

Part A — Routes and handler

A7. Add handler methods

File: `user-service/src/main/java/com/hartford/tms/user/handler/UserHandler.java`. Add imports for the two DTOs, RegistrationRequestResponse, and Flux. Add these methods to UserHandler.

```
public Mono<ServerResponse> requestRegistration(ServerRequest request) {
    return validator.body(request, RegistrationRequestCreate.class)
        .flatMap(userService::requestRegistration)
        .flatMap(body -> ServerResponse.status(HttpStatus.CREATED)
            .contentType(MediaType.APPLICATION_JSON).bodyValue(body));
}

public Mono<ServerResponse> pendingRegistrationRequests(ServerRequest request) {
    return RequestUser.require(request, "ADMIN").flatMap(admin -> {
        int page = request.queryParam("page").map(Integer::parseInt).orElse(0);
        int size = request.queryParam("size").map(Integer::parseInt).orElse(20);
        if (page < 0 || size < 1 || size > 100)
            return Mono.error(new BadRequestException("page must be >= 0 and size must be 1..100"));
        return userService.pendingRegistrationRequests(page, size).collectList()
            .flatMap(items -> ServerResponse.ok().contentType(MediaType.APPLICATION_JSON)
                .bodyValue(items));
    });
}

public Mono<ServerResponse> approveRegistrationRequest(ServerRequest request) {
    return RequestUser.require(request, "ADMIN")
        .flatMap(admin -> userService.reviewRegistrationRequest(
            Params.longVar(request, "id"), true, null, admin.id()))
        .flatMap(this::ok);
}

public Mono<ServerResponse> rejectRegistrationRequest(ServerRequest request) {
    return RequestUser.require(request, "ADMIN").flatMap(admin ->
        request.bodyToMono(RejectRegistrationRequest.class)
            .defaultIfEmpty(new RejectRegistrationRequest(null))
            .flatMap(body -> userService.reviewRegistrationRequest(
                Params.longVar(request, "id"), false, body.reason(), admin.id())))
        .flatMap(this::ok);
}
```

A8. Add reject DTO

New file: `user-service/src/main/java/com/hartford/tms/user/dto/RejectRegistrationRequest.java`

```
package com.hartford.tms.user.dto;
import jakarta.validation.constraints.Size;
public record RejectRegistrationRequest(@Size(max = 500) String reason) {}
```

A9. Register the routes

File: `user-service/src/main/java/com/hartford/tms/user/router/UserRouter.java`. Add these before the existing admin user routes. The request route is public at service-handler level; gateway security is the edge protection. Admin handlers still explicitly require ADMIN.

```
// Public: submit signup request (does NOT create a login account)
.POST("/users/registration-requests", userHandler::requestRegistration)

// Admin-only review (enforced again by RequestUser.require in handlers)
.GET("/admin/registration-requests", userHandler::pendingRegistrationRequests)
.PATCH("/admin/registration-requests/{id}/approve", userHandler::approveRegistrationRequest)
.PATCH("/admin/registration-requests/{id}/reject", userHandler::rejectRegistrationRequest)
```

A10. Update API Gateway security

File: `api-gateway/src/main/java/com/hartford/tms/gateway/config/SecurityConfig.java`. Add the public request path to the permitAll matcher. Do NOT permit /admin/registration-requests.

```
// In authorizeExchange, keep admin matching and add only this exact public route:
.pathMatchers(HttpMethod.POST, "/users/registration-requests").permitAll()
.pathMatchers("/admin/**").hasRole("ADMIN")
.anyExchange().authenticated()
```

File: `api-gateway/src/main/java/com/hartford/tms/gateway/config/SecurityConfig.java`. Also add the exact path to `PUBLIC_PATHS`, otherwise JwtFilter rejects it before authorization rules can permit it.

```
private static final List<String> PUBLIC_PATHS = List.of(
    "/auth/login", "/auth/refresh", "/actuator",
    "/users/registration-requests"
);
```

The filter currently uses startsWith, so using a narrow prefix is safer than adding a broad /users path. For stricter matching, change the filter to compare exact path plus HTTP method, or use a dedicated public-route predicate. Do not add /admin/** or /users/** to the public list.

Part B — Faster task loading (recommended)

The immediate 400 is fixed by keeping size ≤100. The longer delay is partly architectural: the frontend asks for tasks once per project. A single `GET /tasks/visible` is a worthwhile optimization, but it must return only tasks the authenticated caller is allowed to see. Do not implement a raw “all tasks” query without authorization.

B1. Recommended API contract

```
GET /tasks/visible?page=0&size=50&sort=updatedAt,desc
Authorization: Bearer <JWT>

200 response (same PageResponse shape used elsewhere):
{
  "content": [ /* TaskResponse items visible to this user */ ],
  "page": 0, "size": 50, "totalElements": 120,
  "totalPages": 3, "last": false
}
```

Expected access rules: ADMIN can see all tasks; PROJECT_MANAGER can see tasks only in projects they manage; TEAM_MEMBER can see tasks in projects they belong to, with the same visibility policy as the existing project/task screens. Confirm the exact project membership model before implementation.

B2. Files that need changes for a secure endpoint

• `project-service/src/main/java/.../ProjectRouter.java` and its handler/service: add an internal endpoint that returns the project IDs visible to a caller. The existing task service ProjectClient only exposes getProject(id), not a visible-project list.

• `task-service/src/main/java/com/hartford/tms/task/client/ProjectClient.java`: add a WebClient method to call the new internal project endpoint. Pass user ID/role from the gateway-derived RequestUser context; do not trust client-supplied X-User headers.

• `task-service/src/main/java/com/hartford/tms/task/repo/TaskRepo.java`: add a paged query for task rows where project_id is in the authorized project ID list. If the list is empty, return an empty page. Add count query using the same filter.

• `task-service/src/main/java/com/hartford/tms/task/service/TaskService.java`: add visibleTasks(RequestUser, Pageable), obtain authorized project IDs, query page + count, map rows to TaskResponse and PageResponse.

• `task-service/src/main/java/com/hartford/tms/task/handler/TaskHandler.java`: parse pageable with the existing Params.page(...), require authenticated RequestUser, call TaskService.visibleTasks.

• `task-service/src/main/java/com/hartford/tms/task/router/TaskRouter.java`: add .GET("/tasks/visible", taskHandler::visibleTasks) before /tasks/{id}.

Why this guide does not give a blind copy-paste query for project authorization: the supplied TaskService authorizes tasks by fetching ProjectInfoDto for a specific project, and ProjectClient has no endpoint to enumerate all projects visible to a user. A “select all tasks” replacement would expose tasks across tenants/projects. Implement the project-service visible-project lookup first, then query tasks only for those IDs.

B3. Keep pagination safe

```
// Existing file: task-service/src/main/java/com/hartford/tms/task/util/Params.java
// Existing rule is correct: page >= 0 and size between 1 and 100.
// Do not change the upper bound to 200 just to hide frontend errors.
Params.page(request, SORTABLE, "updatedAt,desc", Sort.Direction.DESC);
```

Part C — Loading delay and member list

C1. Do not add blocking calls

This is a Spring WebFlux + R2DBC reactive backend. Keep repository/controller methods returning Mono/Flux; do not call block(), Thread.sleep(), or run synchronous database calls on the event loop. Those patterns make loading worse under concurrent requests.

C2. Member creation path

Existing path: `POST /admin/users` → `UserHandler.createUser` → `UserService.create`. It already checks duplicate email and BCrypt-encodes the password. If the UI spinner remains forever after refresh, inspect the browser Network tab for a pending/failed `GET /admin/users` or `GET /users/directory`, then inspect the User Service logs. Do not create a second member endpoint to work around a frontend spinner.

C3. Add database indexes only after checking query plans

The existing users schema already indexes role and active. Registration requests get a status/time index above. Tasks already have indexes for project/status, assignee, and due date. Avoid indiscriminate indexing; verify query plans and table size first.

Part D — Build and test checklist

■ Run the SQL/schema update in a development database; back up data before changing production.

■ Build user-service and api-gateway with the project's Maven wrapper or Maven command.

■ Submit a signup request without a JWT: expect 201 and a response without password/hash.

■ Try to list/approve requests without a JWT: expect 401. Try with TEAM_MEMBER token: expect 403. Try with ADMIN token: expect success.

■ Approve a pending request: verify one active TEAM_MEMBER row appears in users and the request status becomes APPROVED.

■ Reject a pending request: verify status REJECTED, reviewer/time/reason are recorded, and no user account is created.

■ Test duplicate email submissions and duplicate review clicks; expect a conflict/bad request, not two accounts.

■ For tasks, call /tasks/project/{id}?page=0&size;=100: no size error. Try size=101: expect 400 by design.

■ Only after implementing authorized project ID lookup, test /tasks/visible for ADMIN, PROJECT_MANAGER, and TEAM_MEMBER with projects they should and should not access.

Notes before coding

The snippets are tailored to the supplied code layout and conventions, but they are not a complete modified backend repository. Imports and reactive transaction configuration may need small adjustments to match the exact Spring Boot dependency versions. In particular, make approval atomic and make request retry behavior explicit before deploying. Keep the existing admin-only POST /admin/users route for administrator-created accounts.